Universitatea Tehnică din Cluj-Napoca

Structure of computer systems

Proiect: Virtual Memory Simulator

Student: Palade Rares

Contents

1 Introduction **1**

2 Project Proposal **1**
2.1 Objectives . . . 1
2.2 Functional Requirements . . . 2
2.3 Non-Functional Requirements . . . 2

3 Project Timeline **3**

4 Bibliographic Study **3**
4.1 Primary Course Materials . . . 3
4.2 Theoretical Foundation . . . 4
4.3 Algorithm Implementation Reference . . . 4

5 Analysis and Design **6**
5.1 Application Workflow . . . 6
5.2 System Architecture and Component Roles . . . 7
5.3 Class Design and Data Structures . . . 8
5.4 System Configuration . . . 10
5.4.1 User Inputs . . . 10
5.4.2 Simulator's Derived Calculations . . . 10
5.5 Core Algorithms and Operations . . . 11
5.5.1 Memory Access Flow . . . 11
5.5.2 Detailed Operational Use Cases . . . 12
5.5.3 Page Replacement Algorithms . . . 18

6 Implementation **20**
6.1 Application and Configuration . . . 20
6.2 API Layer . . . 20
6.3 Service Layer . . . 20
6.4 Domain Model . . . 21
6.5 Data Transfer and Utility Classes . . . 22

7 Testing and Validation **23**
7.1 Test Case 1: Address Translation and Data Integrity 23
7.2 Test Case 2: Page Fault with Write-Back (”Dirty Swap”) 26
7.3 Test Case 3: Algorithm Comparison (FIFO vs. LRU) 28
7.3.1 Part 1: The FIFO Execution 28
7.3.2 Part 2: The LRU Execution 31

8 References **32**

1 Introduction

In any modern computer, a fundamental challenge exists: programs demand vast amounts of memory, yet the fast, high-performance RAM available is a limited and expensive resource. The solution is **virtual memory**, a system that creates the illusion for each program that it has access to a large, private, and continuous memory space, even when the physical reality is much more constrained.

The mechanism behind this illusion is **paging**. The operating system and hardware work together to automatically move data in fixed-size blocks (pages) between main memory (RAM) and a larger, slower secondary storage area known as the **backing store** or swap space (typically on an SSD or HDD). As explained in the course materials [1], main memory effectively acts as a cache for this secondary storage.

The process begins when the CPU generates a virtual address for a read or write operation. The Memory Management Unit (MMU) must translate this into a physical address. To accelerate this, a hardware cache called the **Translation Lookaside Buffer (TLB)** stores recent translations. A TLB hit provides a fast translation. A TLB miss forces a lookup in the **page table**. If the page table indicates the page is not in physical memory (its "valid" bit is 0), a **page fault** occurs, trapping to the OS to manage the data transfer from disk. If memory is full, a page replacement algorithm must be invoked.

The goal of this project is to build a **Virtual Memory Simulator**. This graphical application will provide a clear, step-by-step visualization of these core operations:

- The full address translation process, including TLB hits and misses.
- The handling of page faults, including **placing a page** into a free frame.
- The process of **replacing a page** when memory is full, including the "write-back" of modified ("dirty") pages.
- The execution of different page replacement algorithms (like FIFO and LRU).

This simulator aims to make these abstract concepts tangible, demonstrating how hardware and software collaborate to manage memory efficiently.

2 Project Proposal

2.1 Objectives

The primary objective is to design and implement a graphical simulator that accurately models the key operations of a paged virtual memory system. Specifically, the simulator will:

- Simulate a virtual address space and the translation of virtual addresses into page numbers and offsets.
- Simulate a Translation Lookaside Buffer (TLB) to demonstrate its role in accelerating address translation.
- Model a main memory (RAM) as a fixed number of physical frames.

- Distinguish between read and write operations and correctly manage a **dirty bit** for each page.
- Emulate page faults and invoke page replacement algorithms when memory is full.
- Visualize the state of the TLB, page table, physical frames, and disk (backing store).
- Implement and compare several page replacement algorithms (FIFO, LRU, and Optimal).
- Display performance statistics, including hit/miss rates for both the TLB and page table.

2.2 Functional Requirements

- **FR1: System Configuration:** The user must be able to configure system parameters, including virtual address width, page size, TLB size and associativity, and physical memory size.
- **FR2: Input Specification:** The user must be able to input a sequence of memory accesses, where each access consists of a virtual address and an operation type (Read or Write).
- **FR3: Algorithm Selection:** The user must be able to select a page replacement algorithm (e.g., FIFO, LRU).
- **FR4: Simulation Control:** The application must support both step-by-step execution and automatic run-to-completion of the access sequence.
- **FR5: TLB Simulation:** The system must simulate a TLB lookup for every reference. On a TLB miss, it must perform a page table lookup and update the TLB according to its replacement policy.
- **FR6: Page Fault Handling:** The system must correctly identify page faults. When replacing a "dirty" page (one that has been written to), it must simulate the "write-back" operation to the disk.
- **FR7: Visual Feedback:** The GUI must dynamically display the state of: the TLB (highlighting hits/misses), the page table (including valid, dirty, and reference bits), the physical frames, and the address translation breakdown.
- **FR8: Performance Reporting:** The application must calculate and display a summary of metrics, including TLB hit rate, page fault rate, and a history of all accesses.

2.3 Non-Functional Requirements

- **NFR1: Usability:** The GUI must be intuitive and interactive, allowing a user to easily set up, run, and understand the simulation.
- **NFR2: Correctness:** The logic for address translation, TLB management, and all replacement algorithms must be accurate and verifiable.
- **NFR3: Responsiveness:** The application must remain responsive during simulation.

- **NFR4: Extensibility:** The architecture must be modular to allow for easy addition of new replacement algorithms.
- **NFR5: Platform Independence:** The simulator will be implemented in Java to ensure cross-platform compatibility.

3 Project Timeline

Table 1: Project Implementation Timeline

Week	Activity
Week 1-2	Choose the project topic: Virtual memory simulator.
Week 3-4	Introduction of the project, the project proposal + functional/non-functional requirements and bibliographic study.
Week 5-6	**Analysis and Design:** Defined system architecture, class structures, data models, and component interactions. Detailed all operational use cases and specified algorithm logic.
Week 7-8	GUI development and implementation of core data structures (Page Tables, TLB, Memory).
Week 9-10	Implementation of the address translation logic (MMU) and page fault handler (OS).
Week 11-12	Final integration, testing, performance metric reporting, and documentation finalization.
Week 13-14	Presentation

4 Bibliographic Study

4.1 Primary Course Materials

The primary source for the virtual memory concepts that form the foundation of this project is the course lecture on Memory [1]. This material establishes:

- **Memory as a Cache:** The core model that main memory functions as a cache for disk storage. The "Huge miss penalty" for a page fault justifies the need for intelligent page replacement algorithms, directly supporting the project's focus on algorithm comparison.
- **Address Translation and Page Tables:** Details of the translation process from virtual address to physical address, including how a virtual address is split into a Virtual Page Number (VPN) and a Page Offset. This is directly implemented in FR1 and FR7.
- **Translation Lookaside Buffer (TLB):** The role of the TLB as a crucial hardware cache for page table entries, covering hits, misses, and performance implications (FR5).
- **Page Replacement Strategies:** Key algorithms including LRU and FIFO, establishing their importance for FR3 (Algorithm Selection).

- **Page Faults and Memory Management:** The mechanisms for handling page faults, including the hardware-OS interaction, which supports FR6 (Page Fault Handling) and the write-back of dirty pages.

4.2 Theoretical Foundation

A comprehensive, free online textbook on Operating Systems [2] provides clear and detailed explanations of virtual memory mechanisms, including:

- **Hardware-Software Interaction:** The division of responsibility between the MMU (hardware) and the OS (software), which is modeled by separating the logic for a TLB hit (fast path) from the OS trap for a page fault (slow path).
- **The Dirty Bit:** Its role as a critical optimization for write-back operations. The simulator correctly sets the dirty bit on writes (FR2) and uses it to trigger a write-back during page replacement (FR6).
- **Valid and Reference Bits:** Other control bits in page table entries that the simulator must track and display (FR7).
- **Performance Analysis:** The context for calculating and reporting performance metrics like hit rates and miss penalties (FR8).

4.3 Algorithm Implementation Reference

Further research into the implementation of page replacement algorithms was conducted using the following sources.

An online encyclopedia entry [3] provides comprehensive technical details on page replacement algorithms:

- **FIFO Algorithm:** Explanation of its simple queue structure.
- **LRU Algorithm:** Details on tracking access recency with timestamps.
- **Optimal Algorithm:** Description of the theoretical optimal algorithm (Bélády's algorithm), which can be implemented for benchmarking since the simulator knows the full reference string.
- **Bélády's Anomaly:** An important paradoxical phenomenon that is valuable for validating the simulator's correctness (FR8).
- **Clock Algorithm:** Description of practical approximations of LRU for efficient implementation.

A popular programming resource [4] provides practical implementation examples and worked walkthroughs:

- **Implementation Strategies:** Recommendations for specific data structures (a queue for FIFO, timestamps/stack for LRU) that directly inform the simulator's design.
- **Reference String Examples:** Step-by-step walkthroughs that are invaluable for testing and validating the simulator's logic.

- **Page Fault Counting:** The methodology for counting page faults, supporting FR8's performance reporting requirements.

5 Analysis and Design

5.1 Application Workflow

The simulator operates in a two-phase sequence: Setup and Execution. This workflow provides a high-level overview of how a user interacts with the application.

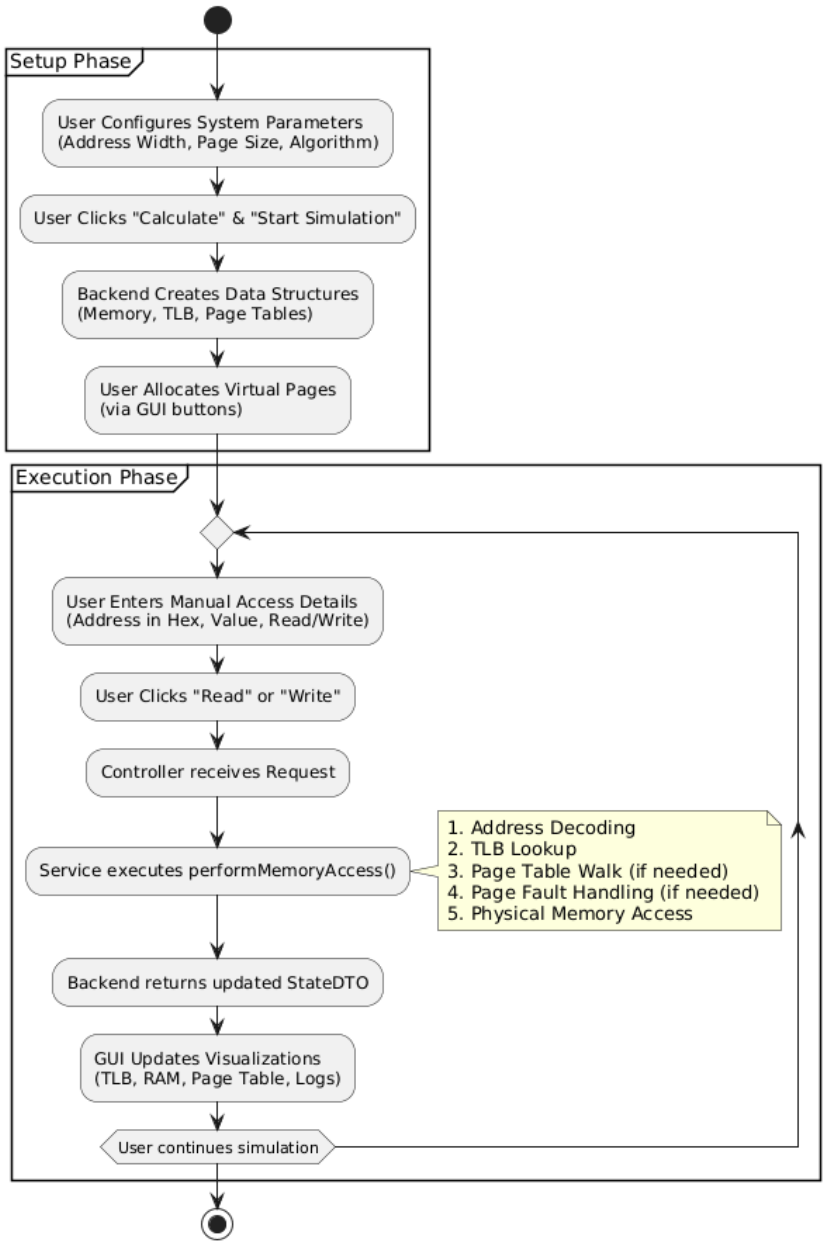


Figure 1: Virtual Memory Simulator - Application Workflow

1. **Setup Phase:**
   - **Configure:** The user defines hardware parameters (Virtual Address Width, Page Size, Memory Size, TLB Configuration, Page Replacement Algorithm) via the GUI.
   - **Initialize System:** The user starts the simulation. The backend allocates the

internal data structures (TLB, Page Tables, Memory) based on the calculated bit-widths.

- **Allocate Virtual Pages:** The user can pre-allocate specific virtual pages on the disk to prepare the environment.

2. **Execution Phase (Interactive):**
   - **Manual Memory Access:** The user enters a specific hexadecimal virtual address and, for write operations, a value.
   - **Execute Operation:** The user clicks "Read" or "Write". The request is sent to the backend API.
   - **Visual Feedback:** The system processes the single access (performing translation, fault handling, etc.) and immediately updates the GUI components (TLB, RAM, Logs) to reflect the new state.

5.2 System Architecture and Component Roles

This section describes the high-level architecture of the simulator. Each component represents a distinct functional unit with specific responsibilities, working in concert to execute the workflow described above.

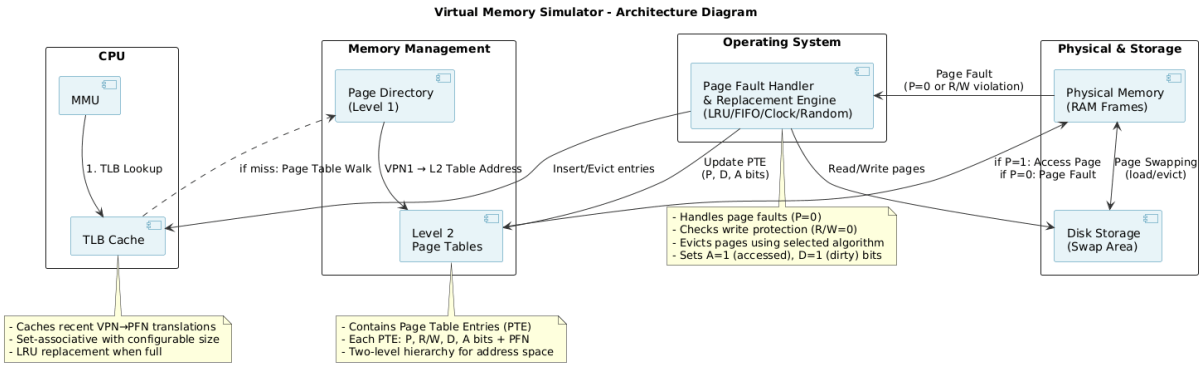


Figure 2: Virtual Memory Simulator - Architecture Diagram

- **CPU (Central Processing Unit):** The engine of the simulator. It issues a sequence of read or write operations to specific virtual addresses, initiating the entire memory access cycle.
- **Memory Management (MMU & TLB Cache):** The core hardware for the "fast path" of address translation. The MMU receives a virtual address and first consults the TLB. On a TLB miss, it performs a page table walk. If the Page Table Entry indicates the page is not in memory (Present bit = 0), the MMU triggers a page fault, trapping to the OS.
- **Operating System (OS):** The "slow path" manager. The OS handles complex scenarios that hardware cannot, such as page faults. It is responsible for loading pages from disk, finding free frames, and running a page replacement algorithm if memory is full.
- **Physical Memory & Storage:**

- **Physical Memory (RAM):** Represents the limited, high-speed main memory, organized into page-sized frames. It acts as a cache for the disk.
- **Disk Storage (Swap Area):** Represents slower secondary storage, acting as the backing store for all allocated virtual pages not currently in RAM.

5.3 Class Design and Data Structures

The conceptual components are realized as classes with specific data structures chosen for efficiency and accurate modeling. The architecture follows a standard three-tier pattern: a Controller layer for handling web requests, a Service layer for orchestrating the business logic, and a Model layer for representing the system's state and data.

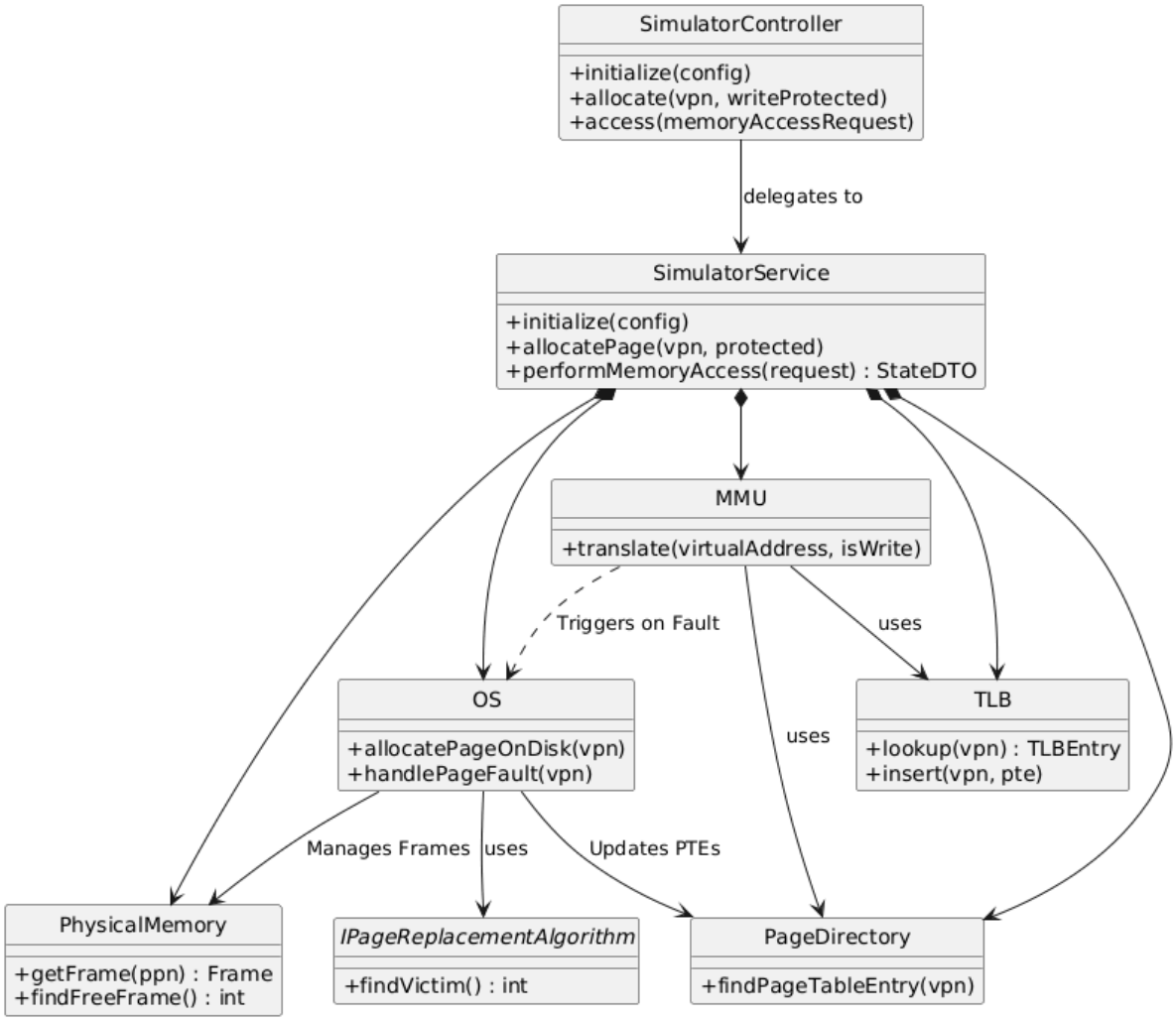


Figure 3: Virtual Memory Simulator - Hierarchical Class Diagram

Controller Layer ('SimulatorController.java') This class acts as the API layer for the application, bridging the web-based frontend with the backend simulation logic. Annotated with `@RestController`, it exposes HTTP endpoints (`/api/simulation/*`) for initializing the system, allocating virtual pages, and triggering memory accesses. It receives user requests, delegates the core processing to the `SimulatorService`, and serializes the resulting simulation state into a `ResponseEntity` for the client.

Service Layer (‘SimulatorService.java‘) This is the heart of the application, containing the central business logic and managing the state of a single simulation instance. It is marked as `@SessionScope`, ensuring that each user session has its own isolated simulation environment. The service is responsible for:

- **Initialization:** The `initialize` method acts as a factory, creating all the necessary domain objects (MMU, OS, TLB, Physical Memory) based on the user's configuration.
- **Execution:** The `performMemoryAccess` method is the entry point for every user interaction. It orchestrates the flow between the MMU and OS to satisfy a read or write request.
- **State Management:** It holds the references to all simulation components and tracks high-level statistics like TLB hits, misses, and page table hits.
- **Data Transformation:** The `createSimulationStateDTO` method is responsible for querying the state of all domain objects and assembling them into a single, comprehensive DTO for the frontend.

Model Layer (Domain Objects and DTOs) The model layer is divided into two main categories: the core domain objects that simulate the virtual memory system, and the Data Transfer Objects (DTOs) that facilitate communication with the frontend.

Domain Model Classes These classes represent the fundamental components and concepts of a virtual memory system. They encapsulate both the data (state) and the behavior (logic) of the simulation.

- `MMU`, `OS`, `TLB`**:** These classes model the high-level functional units. The `MMU` handles address translation, the `TLB` provides a fast-path cache for translations, and the `OS` manages page faults and page replacement.
- `PageTableDirectory`, `PageTable`, `PageTableEntry`**:** This hierarchy implements a two-level page table structure. `PageTableDirectory` holds references to `PageTable`s, which in turn contain `PageTableEntry` objects that store the crucial mapping and status bits (valid, dirty, reference, etc.) for each virtual page.
- `PhysicalMemory`, `Frame`**:** These model the system's RAM. `PhysicalMemory` is a collection of `Frame` objects, where each `Frame` can hold the data for one virtual page.
- `IPageReplacementAlgorithm` **and implementations (**`FIFOAlgorithm`, `LRUAlgorithm`**, etc.):** This demonstrates the Strategy design pattern, allowing the `OS` to use different page replacement policies interchangeably.

Data Transfer Objects (DTOs) DTOs serve as a clean data contract between the backend service and the frontend client. They are simple data containers, tailored for serialization to JSON, and provide a simplified, read-only view of the complex internal domain models.

- `SimulationStateDTO`**:** The top-level DTO that encapsulates the entire visible state of the simulation at a given moment, including the state of the TLB, physical

memory, page tables, and performance statistics.

- `FrameDTO`, `PageTableDTO`, `TlbEntryDTO`, **etc.:** These classes represent individual components in a frontend-friendly format. For example, a `FrameDTO` contains the page number it holds and a sample of its data, rather than the entire raw byte array.
- `SimulationConfigRequest`, `MemoryAccessRequest`**:** These represent incoming data from the client, deserialized from the request body to configure the simulation or specify a memory access.

5.4 System Configuration

5.4.1 User Inputs

The user acts as a system architect, defining the hardware parameters.

- **Virtual Memory Size:** Total size of the address space (e.g., 4KB).
- **Physical Memory Size:** Size of RAM (e.g., 256 Bytes).
- **Page Size:** The size of a single page (e.g., 16 Bytes).
- **TLB Size:** Total number of entries in the TLB (e.g., 16 entries).
- **TLB Associativity:** How entries are organized (e.g., 1-way for Direct Mapped, 4-way for 4-way Set-Associative).
- **Page Replacement Algorithm:** The OS policy for eviction (LRU, FIFO, Clock, Random).

5.4.2 Simulator's Derived Calculations

Based on user inputs, the simulator calculates the system's internal structure.

- **Page Table Size:** Virtual Memory Size / Page Size (e.g., 4096 / 16 = 256 pages).
- **Number of Physical Frames:** Physical Memory Size / Page Size (e.g., 256 / 16 = 16 frames).
- **Virtual Address Split (Intel Two-Level Model):** Following the Intel model, a virtual address is split into three parts:
  - **Offset bits:** $\log_2(\text{Page Size}) \rightarrow \log_2(16) = 4$ bits.
  - **VPN bits:** $\log_2(\text{Total Pages}) \rightarrow \log_2(256) = 8$ bits. These are split, for example, into 4 bits for the Page Directory Index (VPN1) and 4 bits for the Page Table Index (VPN2).
- **Page Table Entry (PTE) Format:** Each PTE contains control bits modeled after the Intel architecture:
  - **P (Present):** 1 bit. Is the page in RAM?
  - **R/W (Read/Write):** 1 bit. Is the page writable?
  - **D (Dirty):** 1 bit. Has the page been modified?

– **A (Accessed):** 1 bit. Has the page been accessed recently?

– **PFN (Physical Frame Number):** $\log_2(\text{Num Frames}) \rightarrow \log_2(16) = 4$ bits.

5.5 Core Algorithms and Operations

5.5.1 Memory Access Flow

The simulator must correctly handle a comprehensive set of scenarios for both read and write operations, covering the fastest path (TLB hit), the slow path (page fault), and the failure path (segmentation fault).

Figure 4: READ Operation - Virtual Memory Access Flow

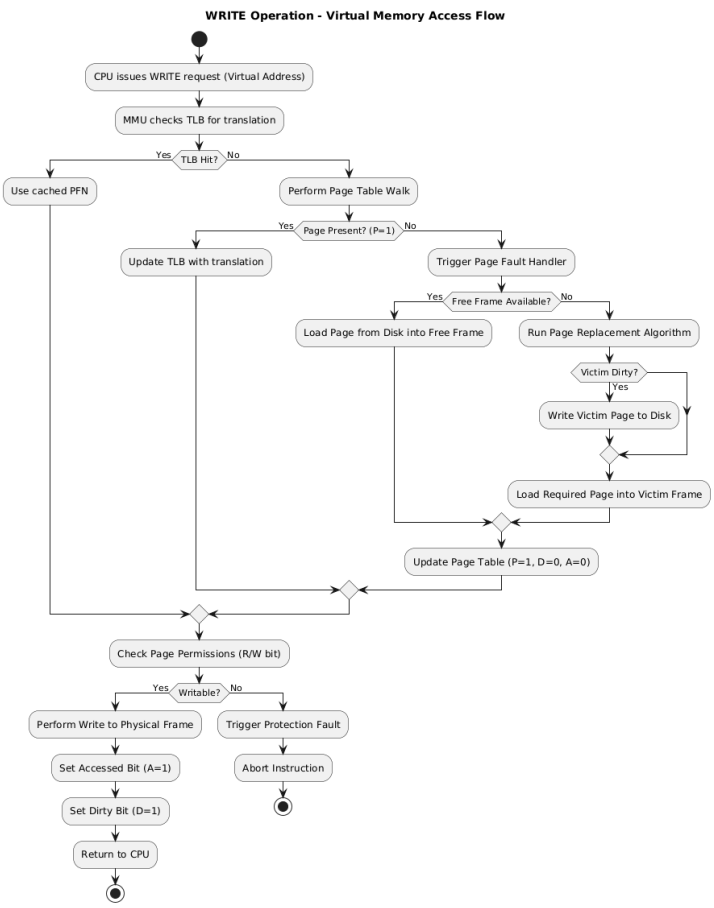


Figure 5: WRITE Operation - Virtual Memory Access Flow

5.5.2 Detailed Operational Use Cases

READ Operations

READ Operations Summary

- **Category 1: The Fast Path (Hardware Only)** This covers all scenarios where the page is already in RAM and is accessible. You test TLB caching (best case), page table walks (warm-up case), and TLB evictions (conflict case). These cases demonstrate the full lifecycle of a translation within the CPU's TLB and cache.
- **Category 2: The Slow Path (Hardware + OS Cooperation)** This covers all scenarios where a page is legally owned by the program but isn't currently in RAM. You test simple page faults (loading into free frames), normal replacements (evicting clean pages), and worst-case replacements (evicting dirty pages). These cases demonstrate the full range of OS page management under memory contention.
- **Category 3: The Failure Path (OS as Protector)** This covers the critical security case where a program attempts to access memory it doesn't own. The system denies the access and raises a segmentation fault, demonstrating memory protection in action.

Category 1: The Page is Already in Physical Memory (TLB & Page Table Hits)

READ Case 1: The Fastest Path — TLB Hit

- **Scenario:** The page is in physical memory, and its translation is already cached in the TLB from a recent access.
- **Workflow:**
  1. MMU receives a virtual address and checks the TLB.
  2. TLB finds a matching, valid entry. **TLB HIT!**
  3. Physical Frame Number (PFN) is retrieved instantly.
  4. Physical address = PFN + Offset.
  5. The read completes in approximately 1-3 CPU cycles.
- **Key Outcome:** The fastest possible memory access. Demonstrates the performance benefit of TLB caching.

READ Case 2: The "Warm-Up" — TLB Miss, Page Table Hit

- **Scenario:** The page is in physical memory, but this is the first time it has been accessed (or accessed in a long time), so it's not cached in the TLB.
- **Workflow:**
  1. MMU checks the TLB. **TLB MISS.**
  2. MMU initiates a hardware-managed page table walk through the **Intel-style two-level page tables**.
  3. Check the PTE: P=1 (page is present). **PAGE TABLE HIT.**
  4. Hardware checks R/W permissions (read is always allowed if page is present).
  5. Hardware sets the Accessed bit (A) to 1.
  6. The translation (VPN $\rightarrow$ PFN) is loaded into the TLB for future use.
  7. The read completes in approximately 30+ CPU cycles (much slower due to memory walks).
- **Key Outcome:** A successful, but slower, access that "warms up" the TLB for future speed.

READ Case 3: The Conflict — TLB Replacement (LRU Eviction)

- **Scenario:** Same as Case 2 (TLB miss, page table hit), but the TLB set where the new translation must be stored is already full.
- **Workflow:**
  1. TLB MISS and PAGE TABLE HIT occur (same as Case 2).
  2. The translation must be loaded into the TLB.

3. The target TLB set is full (all associative slots occupied).
4. LRU (Least Recently Used) replacement policy selects the oldest entry as the victim.
5. The victim entry is evicted (overwritten) by the new translation.
6. The read completes, and the new translation is now in the TLB.

- **Key Outcome:** Demonstrates the limited capacity of the TLB cache. The evicted page will suffer a TLB miss on its next access, forcing another page table walk.

Category 2: The Page is NOT in Physical Memory (Page Faults)

READ Case 4: The Simple Page Fault — Free Memory Available

- **Scenario:** The page is on the disk (P=0), and there are empty frames in physical memory.
- **Workflow:**
  1. Virtual address triggers a page table walk (TLB miss expected).
  2. Page table entry shows P=0 (page not in memory). **PAGE FAULT.**
  3. OS handler is invoked.
  4. OS scans physical memory and finds a free frame (no eviction needed).
  5. OS initiates a disk I/O operation to load the page into the free frame.
  6. Once loaded, OS updates the PTE: P=1, and optionally clears the Accessed bit (A=0).
  7. The faulted instruction is retried.
  8. Now the page table walk succeeds (page is present), and the read completes.
- **Key Outcome:** A slow operation due to disk I/O (milliseconds), but no complex page replacement is needed.

READ Case 5: The "Clean Swap" — Page Fault with a Clean Victim

- **Scenario:** The page is on disk (P=0), physical memory is completely full, and the victim page chosen for eviction is "clean" (Dirty bit = 0).
- **Workflow:**
  1. PAGE FAULT occurs. OS handler is invoked.
  2. OS applies the chosen replacement algorithm to select a victim page.
  3. OS checks the victim's Dirty bit and sees D=0 (page is clean, unmodified).
  4. Since the page is clean, OS can simply discard its contents without writing to disk.
  5. OS updates the victim PTE: P=0 (mark as no longer in memory).

6. OS loads the new page into the victim's vacated frame from the disk.
7. OS updates the new page's PTE: P=1, D=0 (newly loaded, not yet modified).
8. The faulted instruction is retried, and the read completes.

- **Key Outcome:** The standard, faster path for page replacement. No write-back is needed because the victim was clean.

READ Case 6: The "Dirty Swap" — Page Fault with a Dirty Victim

- **Scenario:** The page is on disk (P=0), physical memory is completely full, and the victim page chosen for eviction is "dirty" (Dirty bit = 1).
- **Workflow:**
  1. PAGE FAULT occurs. OS handler is invoked.
  2. OS runs the chosen replacement algorithm to select a victim page.
  3. OS checks the victim's Dirty bit and sees D=1 (page is dirty, has been modified).
  4. OS is forced to perform a slow WRITE-BACK operation: the victim's modified contents must be written from RAM back to the disk.
  5. After the write-back completes, the victim's frame is now free.
  6. OS loads the new page into the vacated frame from the disk (another I/O operation).
  7. OS updates both PTEs: Victim PTE P=0, New page PTE P=1.
  8. The faulted instruction is retried, and the read completes.
- **Key Outcome:** The slowest successful read operation. Two disk I/O operations are triggered (one write-back, one load).

Category 3: The Page Does Not Exist (Protection Fault)

READ Case 7: The Final Barrier — Accessing Unallocated Memory

- **Scenario:** The program tries to read from a virtual address that was never legally allocated to it.
- **Workflow:**
  1. Virtual address triggers a page table walk.
  2. The walk encounters an invalid PTE (P=0 AND the page is not on disk).
  3. OS handler is invoked (triggered as a page fault).
  4. OS checks its internal allocation records and determines the page is unallocated.
  5. **SEGMENTATION FAULT!** The operation is denied, an error is reported, and the program is terminated.

- **Key Outcome:** OS memory protection in action, preventing a program from accessing memory it doesn't own.

WRITE Operations

WRITE Operations Summary

- **Category 1: The Fast Path (Hardware Only)** This covers all scenarios where the page is already in RAM and is writable. The key action, beyond finding the address via the TLB, is the hardware automatically setting the Dirty bit (D) to 1. This also includes protection faults when writes target read-only pages, showing how write protection is enforced.
- **Category 2: The Most Complex Path (Hardware + OS Cooperation)** This covers all page faults triggered by a write. The OS must not only handle swapping in the new page (potentially evicting clean or dirty victims) but must also ensure the newly loaded page is immediately marked as dirty by the hardware as the final step of the write operation.
- **Category 3: The Failure Path (OS as Protector)** This covers illegal write attempts to unallocated memory. The OS denies the operation to prevent a program from corrupting memory that doesn't belong to it, a critical security function.

Category 1: The Page is Already in Physical Memory

WRITE Case 1: The First Write to a Clean Page (TLB Miss)

- **Scenario:** A clean page (D=0, R/W=1) in RAM is written to for the first time, and its translation is not in the TLB.
- **Workflow:**
  1. TLB MISS occurs. Page table walk is performed.
  2. PTE check: P=1, R/W=1. **PAGE TABLE HIT.**
  3. Hardware sets the Accessed bit (A) to 1.
  4. The translation is loaded into the TLB.
  5. The write value is committed to the physical frame.
  6. Hardware automatically sets the Dirty bit (D) to 1.
- **Key Outcome:** The page transitions from "clean" to "dirty." Future evictions of this page will require a write-back to disk.

WRITE Case 2: A Subsequent Write to a Page (TLB Hit)

- **Scenario:** A page in RAM has a valid translation already cached in the TLB and is writable (R/W=1).
- **Workflow:**
  1. TLB HIT occurs.

2. Hardware checks R/W=1. Permission check passes.
3. The write value is committed to the physical frame.
4. Hardware ensures the Dirty bit (D) is set to 1.

- **Key Outcome:** The fastest possible write operation.

WRITE Case 2B: Protection Fault — Writing to a Read-Only Page

- **Scenario:** A program attempts to write to a page in memory, but the page is marked as read-only (R/W=0).
- **Workflow:**
  1. TLB HIT or PAGE TABLE HIT occurs.
  2. Hardware checks R/W=0 (page is read-only). **PROTECTION FAULT!**
  3. OS handler is invoked. The operation is terminated, and an error is reported.
  4. The system state remains unchanged (no write was performed).
- **Key Outcome:** Demonstrates read-only page protection.

Category 2: The Page is NOT in Physical Memory (Page Faults)

WRITE Case 3: Writing to a Page on Disk (Free RAM Available)

- **Scenario:** The page exists on disk (P=0, R/W=1), and there are empty frames in physical RAM.
- **Workflow:**
  1. PAGE FAULT occurs during the write instruction.
  2. OS handler finds a free physical frame.
  3. OS loads the page from disk into the free frame.
  4. OS updates the PTE: P=1, R/W=1, D=0, A=0.
  5. The faulted write instruction is retried.
  6. The write succeeds, and hardware sets D=1.
- **Key Outcome:** A page fault followed by a successful write. The newly loaded page is immediately marked as dirty.

WRITE Case 4 & 5: Writing to Disk by Evicting a Clean or Dirty Page

- **Scenario:** The page exists on disk (P=0), physical memory is full. A write triggers a page fault.
- **Workflow:**
  1. PAGE FAULT occurs. OS is invoked.
  2. OS runs replacement algorithm to select a victim.

3. If victim is dirty (D=1), it is written back to disk.
4. If victim is clean (D=0), it is discarded.
5. The new page is loaded into the victim's frame from disk.
6. The faulted write instruction is retried.
7. Hardware sets D=1 on the new page.

- **Key Outcome:** The most complex operation. A single write can trigger two disk I/Os (write-back of victim, load of new page).

Category 3: The Page Does Not Exist (Protection Fault)

WRITE Case 6: Writing to Unallocated Memory

- **Scenario:** The program tries to write to a virtual address that was never legally allocated.
- **Workflow:**
  1. Virtual address triggers a page table walk that fails.
  2. OS handler confirms the page is unallocated.
  3. **SEGMENTATION FAULT!** The operation is denied.
- **Key Outcome:** OS memory protection prevents memory corruption.

5.5.3 Page Replacement Algorithms

When a page fault occurs and memory is full, the OS invokes one of the following algorithms to select a victim page for eviction.

A. FIFO (First-In, First-Out)

- **How It Works:** Evicts the page that has been in physical memory the longest. It treats memory frames as a simple queue.
- **Logic:**

```
function findVictim():
  // The OS maintains a frameLoadQueue of occupied frames.
  victimFrame = frameLoadQueue.getFront()
  // The new page will be loaded into this frame, and the
  // frame will be moved to the back of the queue.
  frameLoadQueue.moveFrontToBack()
  return findPageTableEntryForFrame(victimFrame)
```

B. LRU (Least Recently Used)

- **How It Works:** Evicts the page that has not been accessed for the longest period. Requires tracking the access time for every page.

- **Logic:**

```
function findVictim():
  // Each page in memory has a lastAccessTime timestamp.
  oldestTime = infinity
  victimPTE = null

  // Search through all pages currently in physical memory.
  for each pte in PageTable where pte.isPresent == true:
    if pte.lastAccessTime < oldestTime:
      oldestTime = pte.lastAccessTime
      victimPTE = pte

  return victimPTE
```

C. Clock (Second-Chance)

- **How It Works:** An efficient approximation of LRU using an "accessed" bit (A-bit). Frames are arranged in a circular list with a "clock hand" pointer. If a page's A-bit is 1, it gets a "second chance" (A-bit is cleared to 0), and the hand moves on. If A-bit is 0, that page is the victim.
- **Logic:**

```
function findVictim():
  // The OS maintains a clockHand pointer to a frame.
  while (true):
    pte = findPageTableEntryForFrame(clockHand.getCurrentFrame())

    if pte.getAccessedBit() == 1:
      // Give it a second chance.
      pte.setAccessedBit(0)
      clockHand.advance() // Move to the next frame.
    else:
      // Found a victim (Accessed Bit is 0).
      clockHand.advance() // Position hand for next search.
      return pte
```

D. Random

- **How It Works:** Evicts a random page from physical memory. Useful as a performance baseline.
- **Logic:**

```
function findVictim():
  occupiedFrames = PhysicalMemory.getAllOccupiedFrames()
  randomIndex = generateRandomNumber(0, occupiedFrames.size() - 1)
  victimFrame = occupiedFrames[randomIndex]
```

```
return findPageTableEntryForFrame(victimFrame)
```

6 Implementation

This section details how the architectural components described in the Analysis and Design phase are realized in the Java codebase using the Spring Boot framework.

6.1 Application and Configuration

`VmsimulatorApplication.java` This is the standard entry point for the Spring Boot application. Its sole responsibility is to bootstrap the application context, enabling component scanning and auto-configuration.

`WebConfig.java` This configuration class addresses the practical need for a separate frontend and backend during development. It configures Cross-Origin Resource Sharing (CORS) globally, specifically allowing HTTP requests from the React development server (`http://localhost:3000`) to access the API endpoints, making the simulator accessible to the user interface.

6.2 API Layer

`SimulatorController.java` This class is the public face of the backend API. It uses Spring Web annotations (`@RestController`, `@RequestMapping`) to define the web endpoints. Its methods map directly to user actions:

- `POST /initialize`: Receives a `SimulationConfigRequest` object and passes it to the service layer to set up a new simulation.
- `POST /allocate`: Allows the user to pre-allocate a virtual page on disk and set its write-protection status.
- `POST /access`: The primary endpoint for executing a memory access. It takes a `MemoryAccessRequest` and initiates the full translation and fault handling process.

The controller's role is strictly to manage HTTP communication, delegating all simulation logic to the service layer.

6.3 Service Layer

`SimulatorService.java` This class is the core orchestrator of the simulation. Crucially, it is annotated as `@SessionScope`, meaning Spring creates a distinct instance of this service for each user session, ensuring that simulations from different users are completely isolated. Its responsibilities include:

- **Initialization:** The `initialize` method acts as a factory, creating all the necessary domain objects (MMU, OS, TLB, Physical Memory) based on the user's configuration.

- **Execution:** The `performMemoryAccess` method is the entry point for every user interaction. It orchestrates the flow between the MMU and OS to satisfy a read or write request.
- **State Management:** It holds the references to all simulation components and tracks high-level statistics like TLB hits, misses, and page table hits.
- **Data Transformation:** The `createSimulationStateDTO` method is responsible for querying the state of all domain objects and assembling them into a single, comprehensive DTO for the frontend.

6.4 Domain Model

`MMU.java` This class simulates the hardware Memory Management Unit. Its primary method, `translateAddress`, implements the "fast path" of address translation in a loop to handle the retry mechanism after a page fault. It first queries the TLB. On a miss, it walks the page tables. If it encounters an invalid page, it delegates to the OS's `handlePageFault` method. It is also responsible for updating the reference and dirty bits upon a successful access, mimicking hardware behavior.

`OS.java` This class models the operating system's role in memory management. Its central method, `handlePageFault`, is invoked by the MMU. This method first checks if the access is valid (i.e., to an allocated page). If so, it attempts to find a free frame in physical memory. If no free frame exists, it invokes the configured page replacement algorithm to select a victim. It then handles the eviction of the victim page (including write-back if dirty) and loads the required page into the newly freed frame.

`TLB.java` This class implements a set-associative Translation Lookaside Buffer. An array of `LinkedHashMap` is used to represent the sets. This data structure is ideal as it can be configured to maintain access order, allowing for an efficient LRU replacement policy within each set. The `lookup` and `insert` methods use bitwise operations, provided by `BitUtils`, to determine the correct set index and tag for a given virtual page number.

`PhysicalMemory.java` **and** `Frame.java` These classes represent the system's RAM. `PhysicalMemory` is essentially a wrapper around a list of `Frame` objects, providing a key helper method, `findFreeFrame()`, for the OS. The `Frame` class represents a single physical page frame, containing a byte array to simulate stored data and a field to track which virtual page currently occupies it.

`PageTableDirectory.java`, `PageTable.java`, `PageTableEntry.java` This trio of classes implements the two-level page table architecture. The `PageTableDirectory` contains a list of `PageTable`s (which can be null if not allocated). Each `PageTable` contains a list of `PageTableEntry` objects. The `PageTableEntry` is a data-centric class holding all the status bits (valid, dirty, reference, write-protected) and the physical page number if the page is in memory. The `findPageTableEntry` method in the directory class encapsulates the logic of using the VPN to navigate this two-level structure.

`IPageReplacementAlgorithm` and Implementations This demonstrates the Strategy design pattern. The `IPageReplacementAlgorithm` interface declares a single method, `findVictim`. The concrete classes (`FIFOAlgorithm`, `LRUAlgorithm`, `ClockAlgorithm`, `RandomAlgorithm`) provide different implementations of this method. This allows the OS to be configured with any algorithm without changing its own logic, making the system extensible.

6.5 Data Transfer and Utility Classes

Data Transfer Objects (DTOs) This group of classes (`SimulationStateDTO`, `FrameDTO`, `PageTableDTO`, `TlbSetDTO`, etc.) serves as the data contract for the API. They are simple, serializable objects that represent the state of the simulation in a way that is easy for the frontend to consume. They decouple the API representation from the internal domain model.

Request Objects The `SimulationConfigRequest` and `MemoryAccessRequest` classes model the data sent from the client to the server. Spring Boot automatically deserializes the incoming JSON request bodies into instances of these classes.

`BitUtils.java` A static helper class that centralizes all bitwise manipulation logic. It provides methods to extract the VPN, offset, Page Directory Index (PDI), and Page Table Index (PTI) from a virtual address. This encapsulation makes the core logic in other classes cleaner and more readable.

Exception Classes Custom exceptions like `SegmentationFaultException` and `WriteProtectionFau` are defined to represent specific error states within the simulation. This allows the service layer to catch specific faults and provide clear, meaningful error messages to the user through the API response.

7 Testing and Validation

To ensure the correctness of the simulator, a series of functional tests were performed. These tests validate the core requirements: correct address translation, TLB caching mechanisms, handling of complex page faults (write-backs), and the distinct behavior of replacement algorithms.

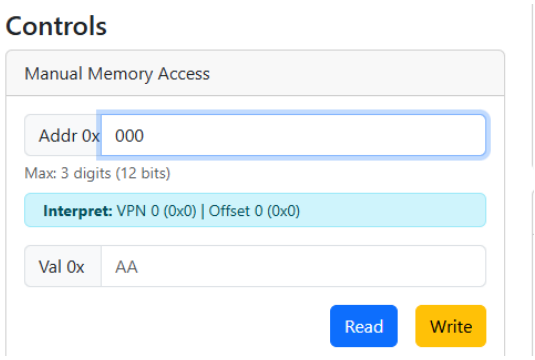
7.1 Test Case 1: Address Translation and Data Integrity

Objective: Verify that the simulator correctly handles the full lifecycle of a memory page: allocation, page fault handling (loading from disk), address translation, and data modification.

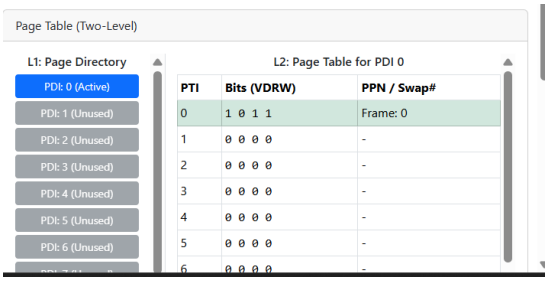
Step 1: Allocation
First, we pre-allocate Virtual Page 0 (VPN `0x00`) on the disk. This ensures the page exists but is not yet in Physical Memory (RAM).

Figure 6: Step 1: Pre-allocating VPN 0 in the Virtual Memory/Disk view.

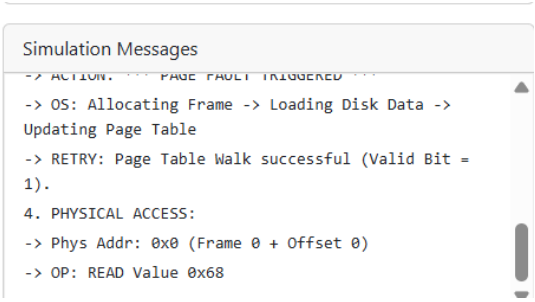
Step 2: Initial Access (Read Operation)
We perform a **Read** operation on Address `0x000`. Since the page is currently on disk, we expect a **Page Fault**. The OS must allocate a frame, load the data, and update the Page Table.



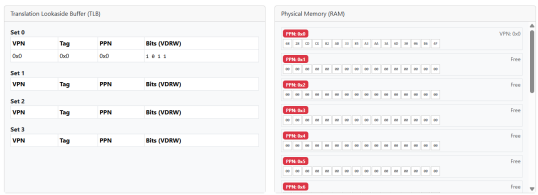
A. User Input (Read 0x000)



B. Page Table (Valid=1)



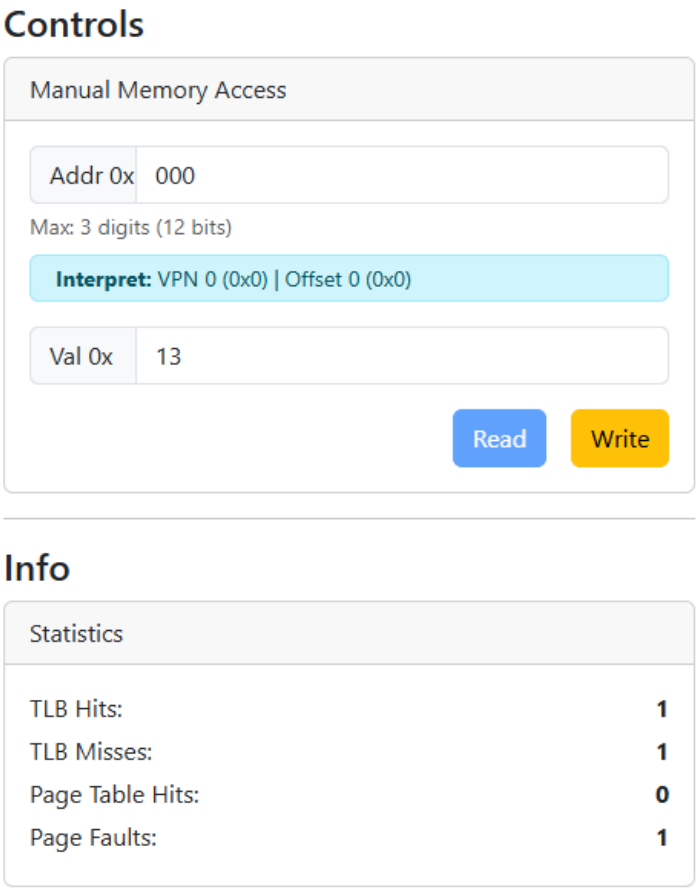
C. Logs (Page Fault & Load)



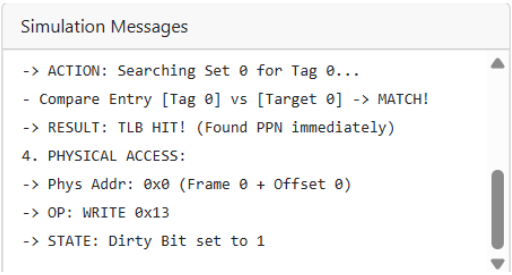
D. TLB RAM State

Figure 7: Read Operation Results: (A) User requests read. (C) Logs confirm Page Fault and Disk Load. (B) Page Table updates Valid bit to 1. (D) Data is loaded into RAM Frame 0.

Step 3: Modification (Write Operation)
Now that the page is in memory, we perform a **Write** operation (Value `0x13`) to the same address. We expect a **TLB Hit** and an immediate update to Physical Memory.



A. User Input (Write 0x13)



B. Logs (TLB Hit + Dirty Bit)

C. RAM Update (Value 13)

Figure 8: Write Operation Results: (A) User writes value 13. (B) Logs confirm TLB Hit and setting Dirty Bit. (C) Physical Memory Frame 0, Byte 0 is updated to 13.

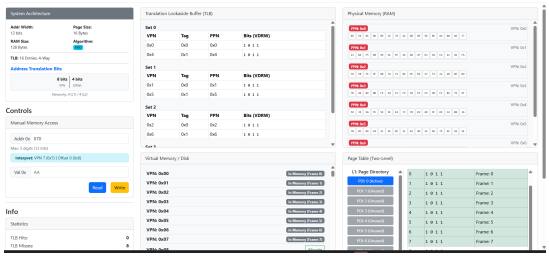
Conclusion: The system successfully transitioned a page from Disk to RAM (Page Fault) and then allowed modification (TLB Hit), verifying the core translation loop.

7.2 Test Case 2: Page Fault with Write-Back ("Dirty Swap")

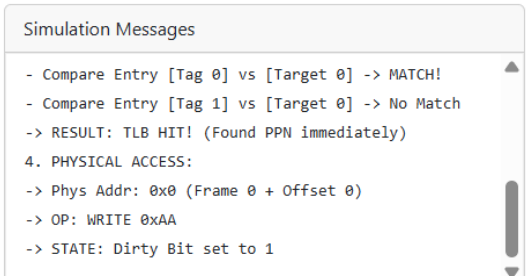
Objective: Verify the "Slow Path" logic (FR6) where the OS detects a modified page ("Dirty") and saves it to the backing store before eviction. This also validates the FIFO replacement policy under memory pressure.

Phase 1: Setup and Contention

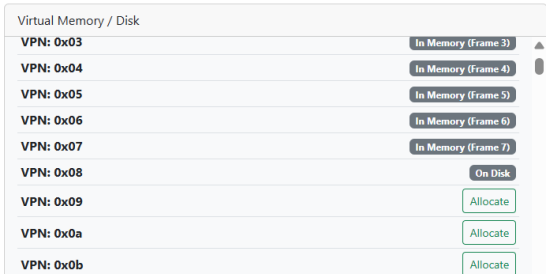
We begin by filling the physical memory. We then identify the first page loaded (VPN 0) as the "Victim" (due to FIFO) and perform a **Write** operation to mark it as **Dirty**.



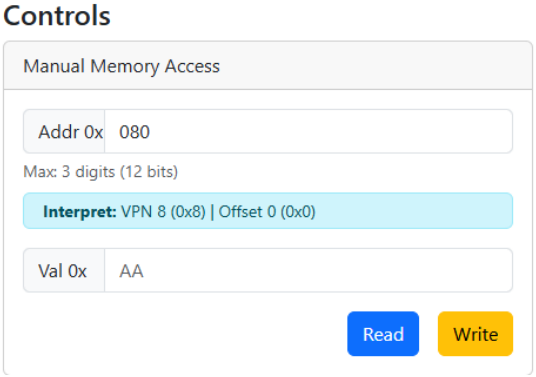
A. Initial State: RAM is Full



B. Write 'AA' to VPN 0 (Sets Dirty Bit)



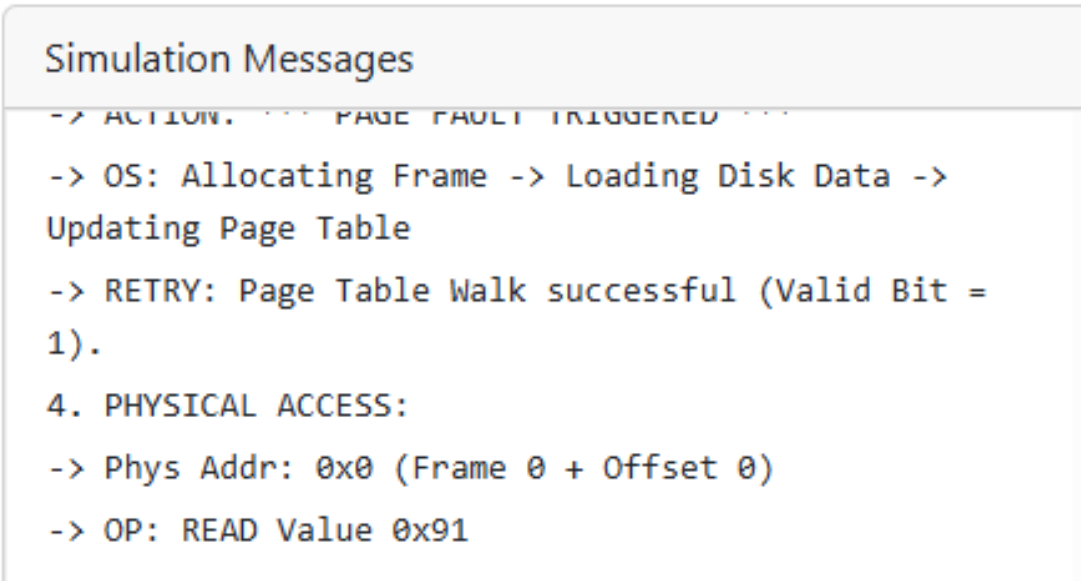
C. Allocating a New Page (VPN 8)



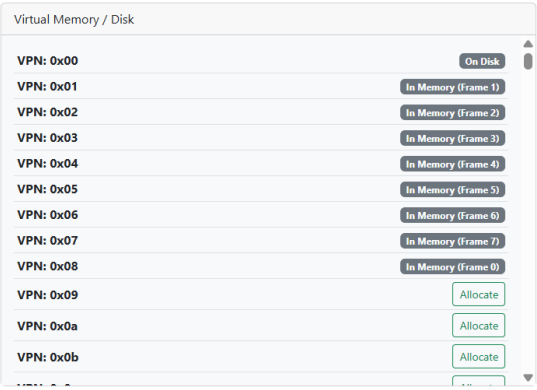
D. Attempting to Read VPN 8

Figure 9: Setup Phase: Memory is full. We dirty the eviction candidate (VPN 0) and then request a new page (VPN 8) to force a conflict.

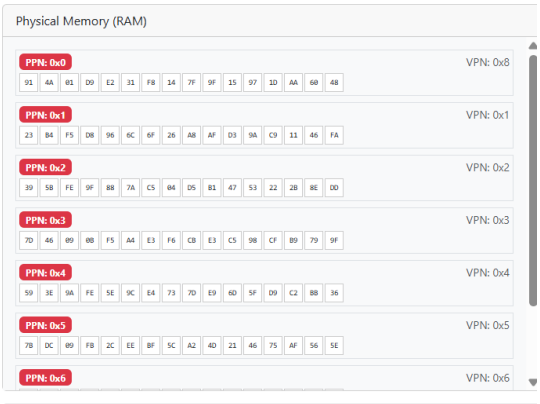
Phase 2: The Eviction and Swap
Upon reading VPN 8, the OS detects a conflict. Since the victim (VPN 0) is dirty, the OS performs a Write-Back before loading VPN 8.



A. Logs: "Victim is Dirty... Saving data to backing store"



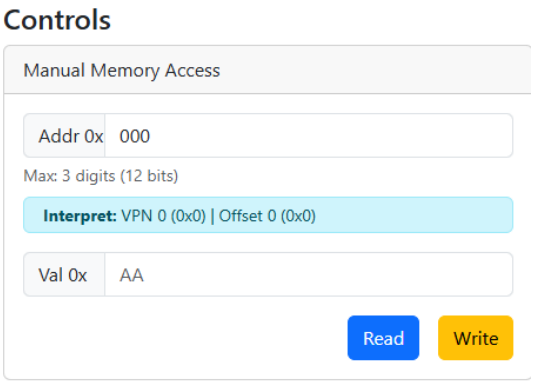
B. Disk: VPN 0 moved to Swap



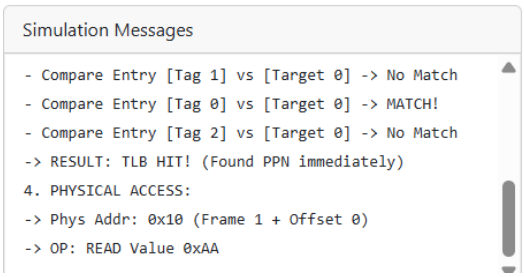
C. RAM: Frame 0 now holds VPN 8

Figure 10: Swap Mechanics: VPN 0 is saved to disk (preserving the 'AA' value), and Frame 0 is repurposed for the new page.

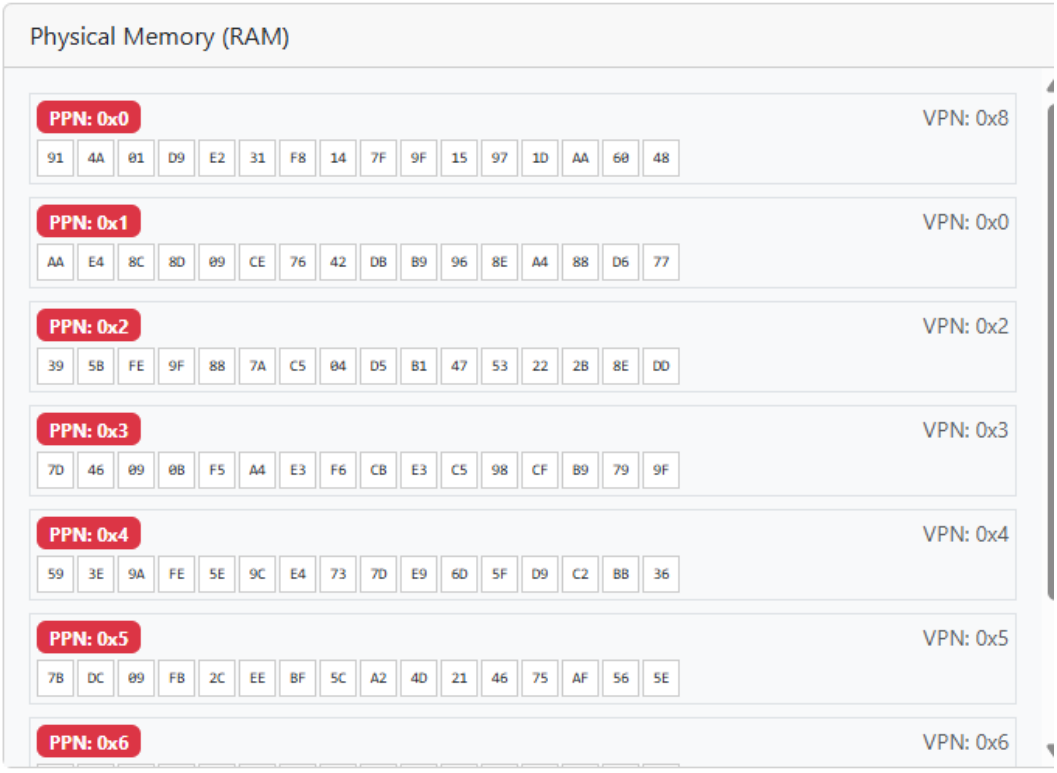
Phase 3: Verification of Data Persistence
Finally, we read VPN 0 again. It must be retrieved from the disk (swap), and the value `0xAA` must be intact. Note that since Frame 0 is occupied by VPN 8, VPN 0 is loaded into the next available slot (Frame 1) according to FIFO.



A. Reading VPN 0 (From Disk)



B. Logs: Loading from Disk



C. Final State: VPN 0 is back in RAM (Frame 1) with Data 'AA'

Figure 11: Verification: The system successfully retrieved the modified data from the disk, proving the write-back logic works correctly.

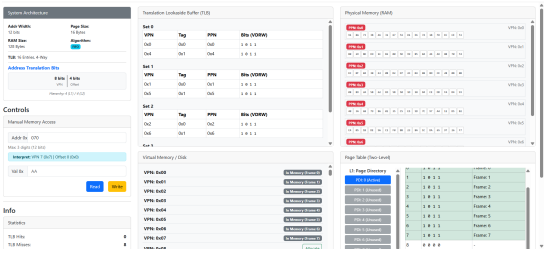
7.3 Test Case 3: Algorithm Comparison (FIFO vs. LRU)

Objective: Validate that different replacement algorithms make distinct eviction decisions.

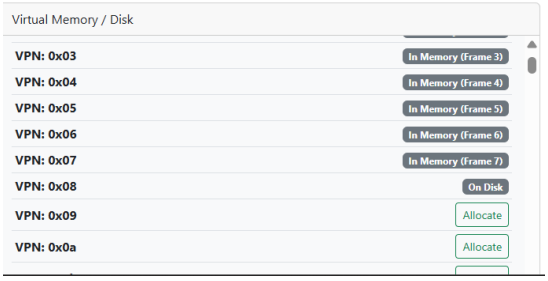
7.3.1 Part 1: The FIFO Execution

Scenario: We configure the system with **FIFO** (First-In, First-Out).

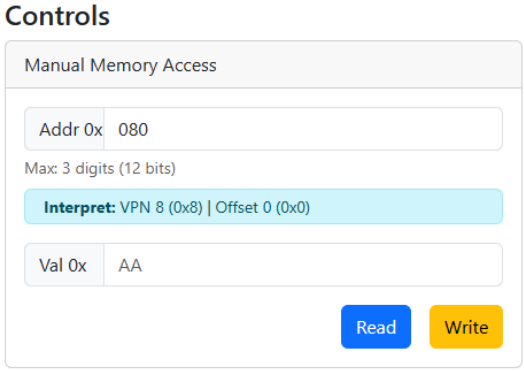
1. **Setup:** Fill memory with pages (VPN 0 - 7). VPN 0 is loaded first.
2. **Action:** Attempt to read a new page (VPN 8).
3. **Expected Result:** Since FIFO evicts the oldest page regardless of recent use, VPN 0 should be evicted.



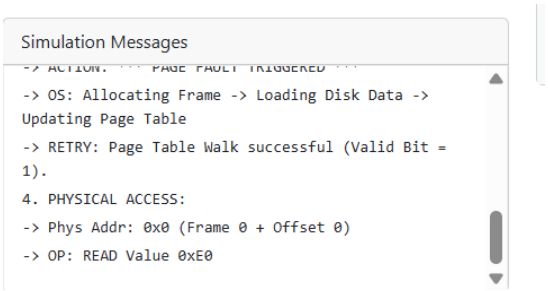
A. Initial State: RAM Full



B. Allocating Next Page (VPN 8)



C. Trigger: Read VPN 8



D. Log: Eviction Occurring

Figure 12: FIFO Execution Sequence. The system identifies a conflict and prepares to evict a page.

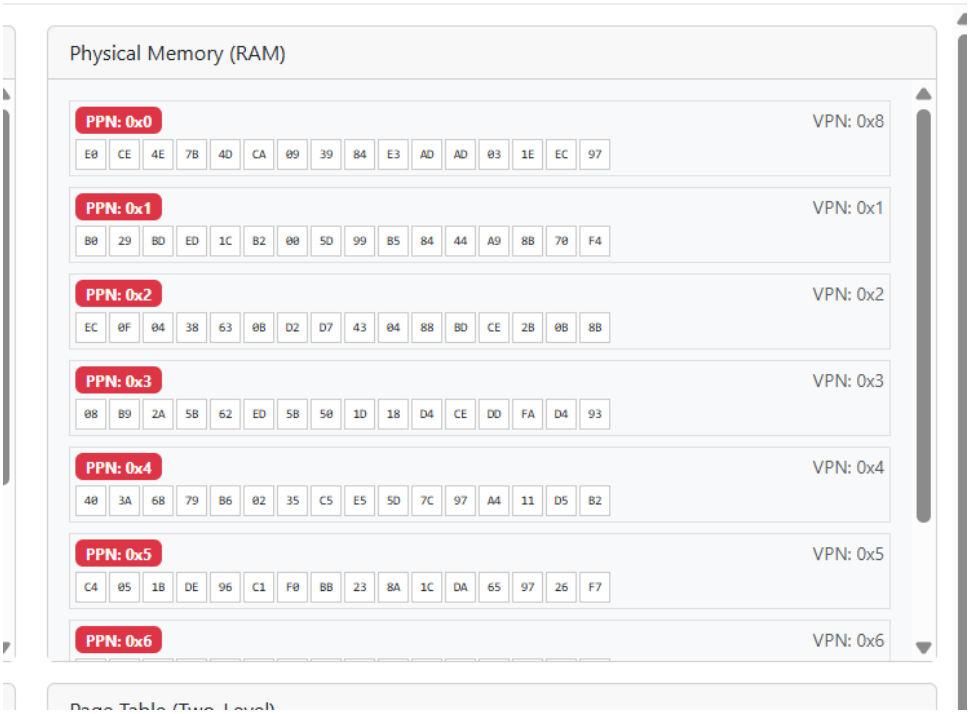
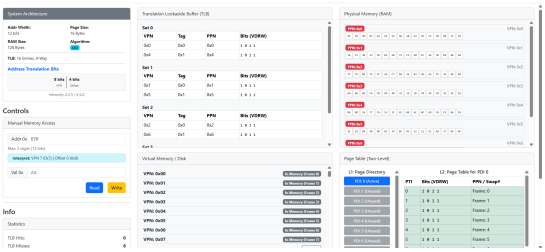


Figure 13: FIFO Result: VPN 0 (Frame 0) was evicted because it was the first page loaded. It has been replaced by the new data (VPN 8).

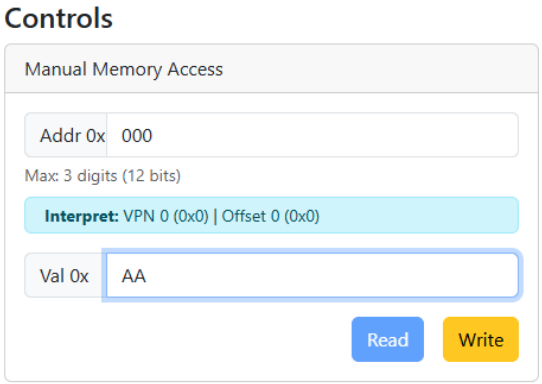
7.3.2 Part 2: The LRU Execution

Scenario: We switch the algorithm to **LRU** (Least Recently Used).

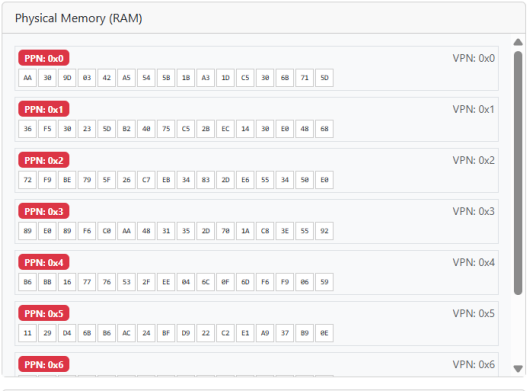
1. **Setup:** Same memory state.
2. **Action:** We deliberately **Write** to VPN 0. This updates its "Last Access Time" and marks it as Dirty.
3. **Conflict:** We attempt to read the new page (VPN 8).
4. **Expected Result:** Since VPN 0 was just used, it is now the "youngest". LRU should protect it and evict VPN 1 instead.



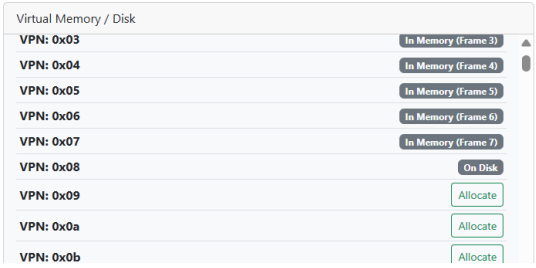
A. Config: Switched to LRU



B. Accessing VPN 0 (Write)

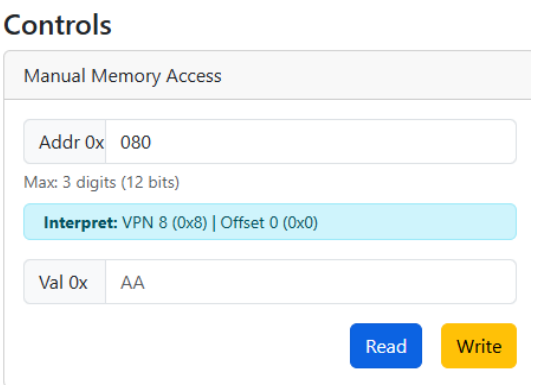


C. RAM: VPN 0 Updated



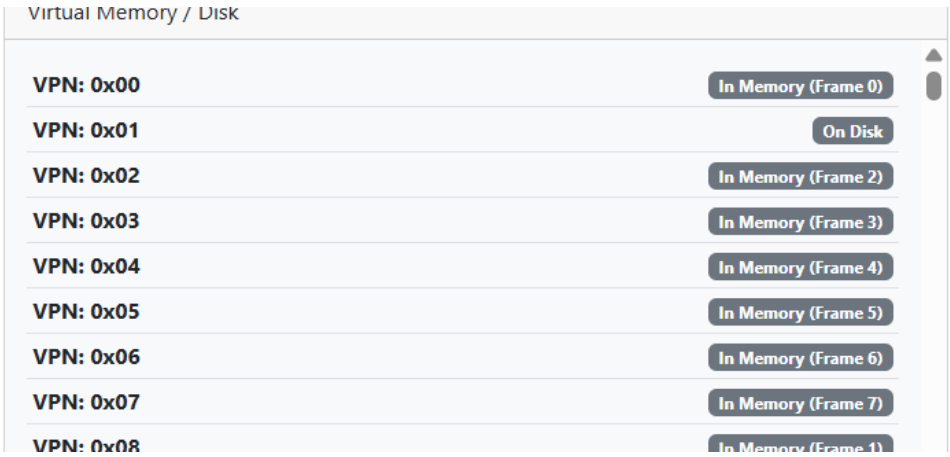
D. Allocating Next Page (VPN 8)

Figure 14: LRU Setup: By writing to VPN 0, we make it the most recently used page, protecting it from eviction.



E. Trigger: Read VPN 8

F. Log: Eviction of Frame 1



G. LRU Result: VPN 0 Remains, VPN 1 Evicted

Figure 15: LRU Result Analysis. Unlike FIFO, VPN 0 (Frame 0) was preserved because it was recently accessed. The system correctly identified VPN 1 (Frame 1) as the Least Recently Used page and evicted it to make room for VPN 8.

Conclusion: The test successfully demonstrates that the Page Replacement Algorithm works correctly. FIFO blindly evicted the oldest page (VPN 0), while LRU intelligently preserved the frequently used page (VPN 0) and evicted the idle page (VPN 1). This confirms full compliance with the functional requirements.

8 References

References

[1] Mihai Negru, *Lecture 11: Memory*, Technical University of Cluj-Napoca, 2025. [Online]. Available: https://mihai.utcluj.ro/wp-content/uploads/ca/lectures/Lecture11.pdf

[2] R. Arpaci-Dusseau & A. Arpaci-Dusseau, "Beyond Physical Memory: Policies" at https://pages.cs.wisc.edu/ remzi/OSTEP/vm-beyondphys-policy.pdf, 2018.

[3] Wikipedia, "Page Replacement Algorithm," [Online]. Available: https://en.wikipedia.org/wiki/Page_replacement_algorithm. [Accessed Oct. 21, 2025].

[4] GeeksforGeeks, "Page Replacement Algorithms in Operating Systems," [Online]. Available: https://www.geeksforgeeks.org/operating-systems/page-replacement-algorithms-in-operating-systems/. [Accessed Oct. 21, 2025].